

# Majority Vote

Abdullah Khaled

|                                            |          |
|--------------------------------------------|----------|
| <b>Design Brief.....</b>                   | <b>2</b> |
| <b>Truth Table &amp; Karnaugh Map.....</b> | <b>3</b> |
| <b>Boolean Algebra Simplification.....</b> | <b>3</b> |
| <b>Circuit Schematics.....</b>             | <b>4</b> |
| <b>Circuit Design Matrix.....</b>          | <b>5</b> |
| <b>AOI Breadboard Prototype Media.....</b> | <b>5</b> |
| Circuit Picture.....                       | 5        |
| Working Circuit Video.....                 | 5        |
| <b>PLD Media.....</b>                      | <b>6</b> |
| Multisim AOI Circuit Schematic.....        | 6        |
| AOI Breadboard Prototype Image.....        | 6        |
| AOI Breadboard Prototype Video.....        | 6        |
| <b>Reflection.....</b>                     | <b>7</b> |

# Design Brief

---

Client: Board of Directors

---

Designer: Abdullah Khaled

---

Problem Statement: Recent elections have made the Board of Directors worried about possible vote miscounting. As such, instead of using paper votes, they wish to use electronic votes to ensure everyone is accounted for.

---

Design Statement: The client wishes the designer to create an electronic voting system that determines whether a majority has been reached and uses the President's vote as a tiebreaker.

---

Constraints:

- Only able to use 2-input AND, 2-input OR, 2-input NAND, and/or 2-input NOR gates
- Must display the pass/fail status of each board member's vote

---

Deliverables:

The final report should include documentation of the following items:

- Design brief
- Definition of this circuit's inputs/outputs
- Truth table
- Unsimplified logic expression
- Karnaugh map
- Steps of Boolean algebra
- Simplified logic expression
- Simplified AOI circuit implementation (Multisim)
- NAND circuit implementation (Multisim)
- NOR circuit implementation (Multisim)
- Photo of AOI breadboard prototype
- Photo of PLD circuit implementation (Multisim) and prototype

---

## Truth Table & Karnaugh Map

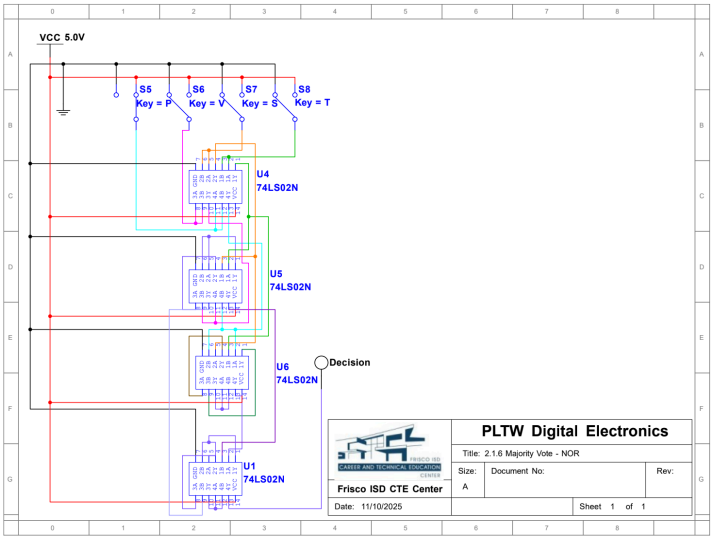
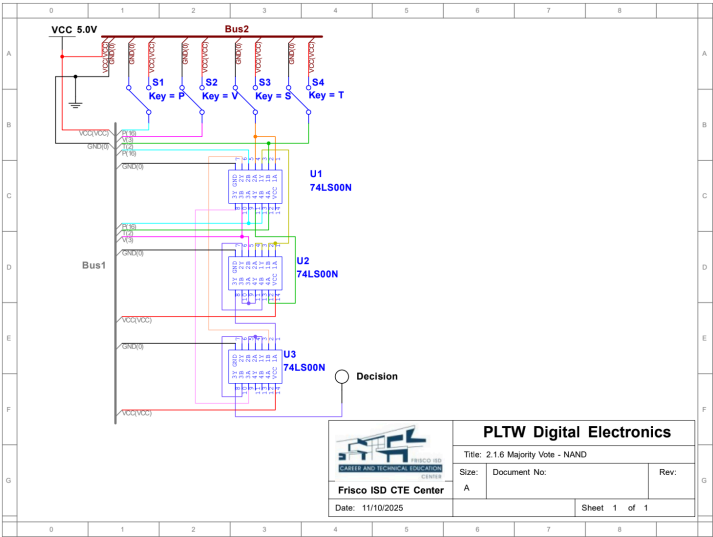
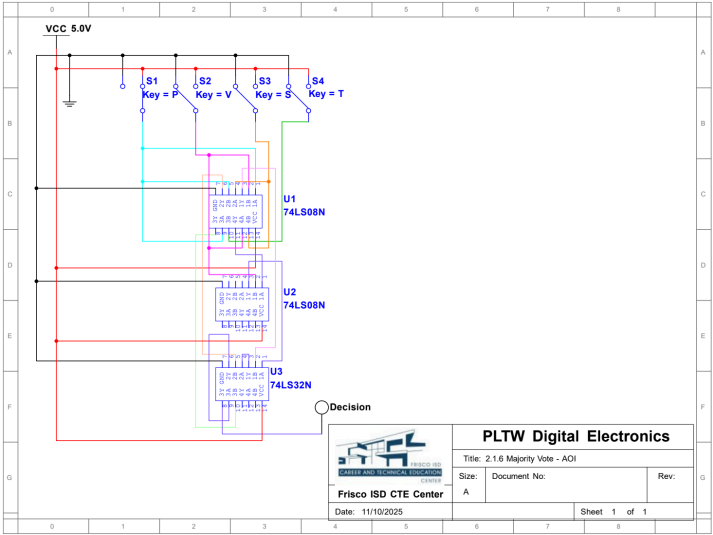
| P | V | S | T | D |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

|                  | $\bar{S}\bar{T}$ | $\bar{S}T$ | $ST$ | $S\bar{T}$ |
|------------------|------------------|------------|------|------------|
| $\bar{P}\bar{V}$ | 0                | 0          | 0    | 0          |
| $\bar{P}V$       | 0                | 0          | 1    | 0          |
| $PV$             | 1                | 1          | 1    | 1          |
| $P\bar{V}$       | 0                | 1          | 1    | 1          |

## Boolean Algebra Simplification

$$\begin{aligned}
 & PVST + PV\bar{S}\bar{T} + PV\bar{S}T + PV\bar{S}\bar{T} + P\bar{V}ST + P\bar{V}\bar{S}\bar{T} + P\bar{V}\bar{S}T + P\bar{V}ST \\
 &= PVS(T+\bar{T}) + PVS(T+\bar{T}) + PVS(T+\bar{T}) + P\bar{V}\bar{S}\bar{T} + P\bar{V}ST \\
 &= PV(S+\bar{S}) + P\bar{V}S + P\bar{V}\bar{S}\bar{T} + P\bar{V}ST \\
 &= PV + P\bar{V}(S + \bar{S}T) + P\bar{V}ST \\
 &= PV + P\bar{V}(S + T) + P\bar{V}ST \\
 &= PV + P\bar{V}S + P\bar{V}T + P\bar{V}ST \\
 &= P(V + \bar{V}T) + P\bar{V}S + P\bar{V}ST \\
 &= P(V + T) + P\bar{V}S + P\bar{V}ST \\
 &= PV + PT + P\bar{V}S + P\bar{V}ST \\
 &= P(V + \bar{V}S) + PT + P\bar{V}ST \\
 &= PV + PS + PT + P\bar{V}ST \\
 &= V(P + \bar{P}ST) + PS + PT \\
 &= V(P + ST) + PS + PT \\
 &= \mathbf{PV + PS + PT + VST}
 \end{aligned}$$

# Circuit Schematics



## Circuit Design Matrix

|                             | Simplified AOI | NAND   | NOR    |
|-----------------------------|----------------|--------|--------|
| Gate Count                  | 8              | 11     | 16     |
| IC Count                    | 3              | 3      | 4      |
| Cost per IC (USD)           | \$0.76         | \$0.33 | \$0.41 |
| Total cost per circuit      | \$2.27         | \$0.99 | \$1.64 |
| Max Propagation Delay* (ms) | 40             | 40     | 60     |

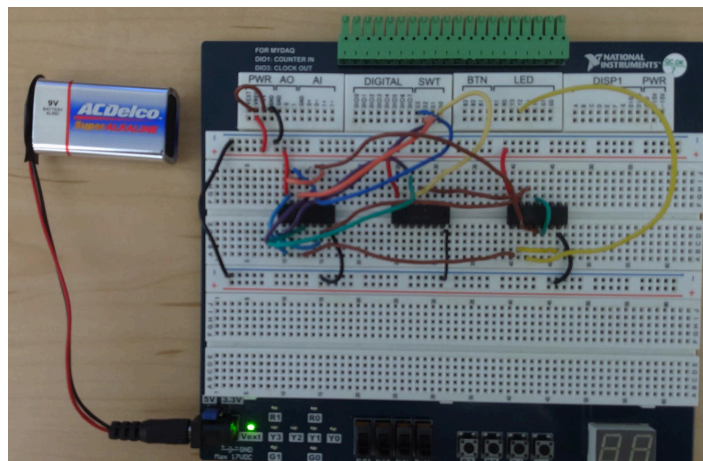
\*Assumes 10 ms per logic gate.

The output “Decision” signifies whether the vote will pass or fail. In logic terms, it outputs the value of the truth table the circuit is modeled after (assuming the circuit is designed correctly).

I would choose to implement the simplified AOI circuit because despite the higher cost than the NAND or NOR circuits, the max propagation delay for the circuit is half of the other two options. Also, the circuit’s gate count is lower and the circuit has the same number of ICs as the NAND gate circuit. The gate count is especially significant since we will be assembling the circuit manually, so decreasing the wiring complexity directly correlates with a decreased chance for mistakes during assembly.

## AOI Breadboard Prototype Media

### Circuit Picture

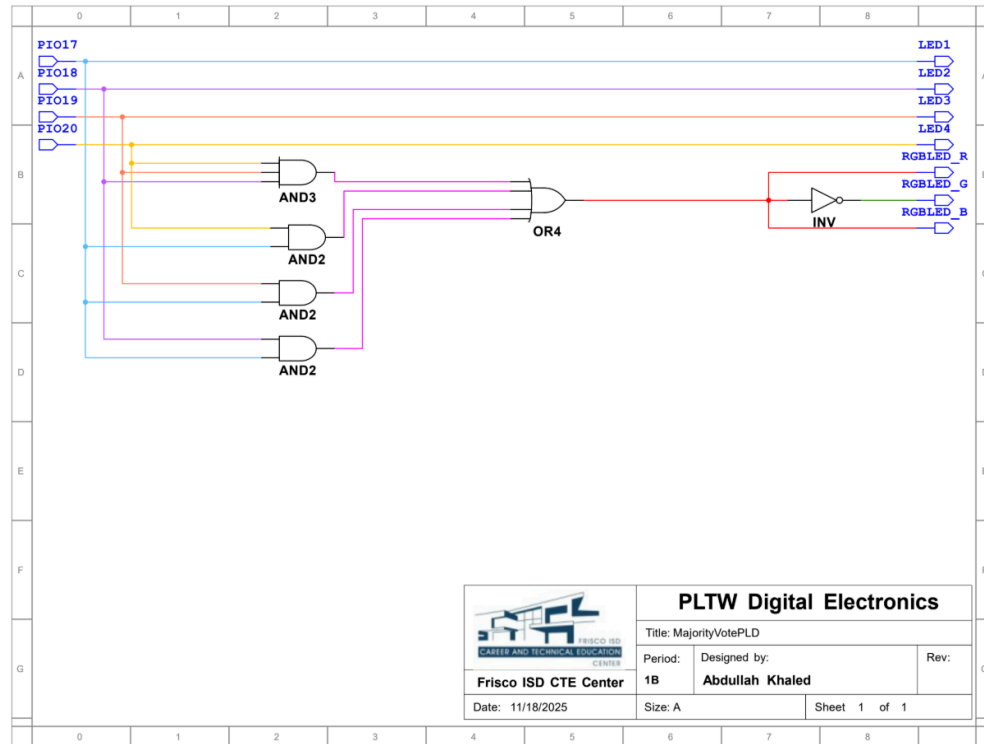


### Working Circuit Video

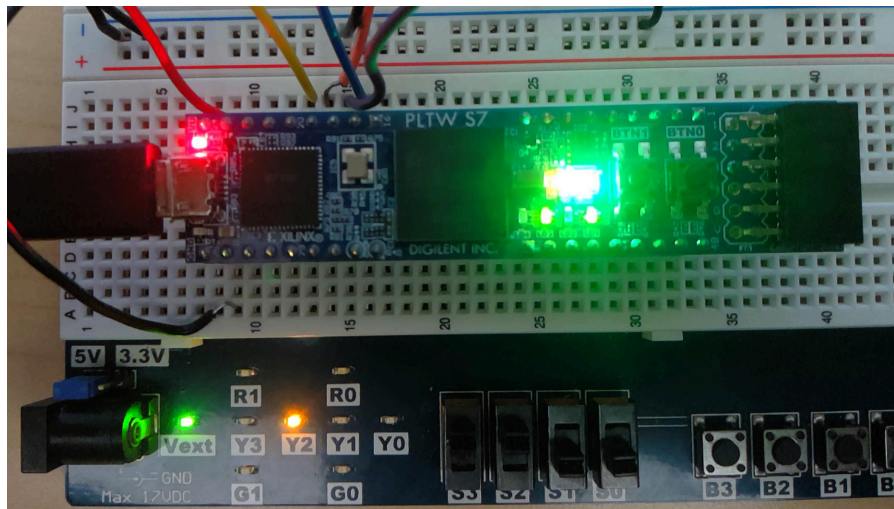
<https://youtu.be/Vcxdboo5JB0>

## PLD Media

### *Multisim AOI Circuit Schematic*



### *AOI Breadboard Prototype Image*



### *AOI Breadboard Prototype Video*

<https://youtu.be/hPqy3gH22aA>

## Reflection

As stated in the design brief, the majority vote is meant to accomplish the task of mitigating potential counting errors in voting systems. In this project, the designer was meant to create a circuit that incorporated both a majority voting system and a presidential overrule in the event of a tied vote. In a real-world context, this circuit could be used in legislative branches such as the Senate. Since the Senate has 100 members, a tied vote would force the Vice President to vote on the issue as well, mimicking the designed circuit (albeit at a much larger scale). In practice, however, the circuit could be easily expanded to accommodate the larger voting body, highlighting one benefit of the design.

One of the larger design challenges the designer faced was the schematics of the simplified circuit. While in theory, the circuit was relatively simple, creating various solutions was difficult. The issue was compounded due to efforts to simplify the circuits, forcing the designer to think critically about circuitry pathing and what connections could be simplified for all circuit types (i.e. AOI, NAND, NOR). Moreover, the designers' desire to utilize the schematic as an assembly guide for the final circuit necessitated both color coding and the use of ICs rather than gates during schematic creation, dramatically increasing the complexity of circuit wiring and organization.

Yet, the designer learned a wide variety of technical skills from the project. Foremost, the designer gained experience and fluency in schematic creation using Multisim. Additionally, the designer furthered his understanding of logic expressions and the simplification process of complex expressions extracted from truth tables using multiple methods, including Karnaugh mapping and boolean algebra simplification. Practically, the designer gained experience in breadboarding more complex circuits, gaining skills that are transferrable to a wide range of projects.

The PLD (Programmable Logic Device) implementation differed significantly from the discrete logic implementation, namely in its complexity. The discrete logic implementation would likely be more suitable for production-level applications, where thousands—if not millions—of devices are made and assembly time is incredibly optimized. However, assembly of circuits even at the scale of complexity found in the project were extremely tedious and time-consuming, a major factor to consider when the production scale of the project is one unit; if a method takes significantly less time, then it likely should be preferred. The PLD meets these criteria by allowing the designer to program the logic using a software tool such as NI Multisim, then upload the logic to the device—hence the name “Programmable Logic Device.” From there, the designer only needs to power the PLD through USB or via the power connectors on the board and provide the PLD with inputs through the respective pins, and the circuit is ready for testing. More so than simply greatly simplifying wiring, though, the PLD allows for rapid prototyping, since the designer can update and upload the logic directly to the device, decreasing iteration time significantly. Overall, the designer will likely consider utilizing PLDs in future projects due to their ease of use and time efficiency when designing and testing circuits.